Lycée Duhamel du Monceau. Niveau : Première.

Numérique et Sciences Informatiques

Traitement de données en tables. Correction.

Dans de nombreuses situations, les données que nous voulons étudier ne sont pas dans des formats simples à traiter. Il arrive fréquemment que des données soient stockées sous forme de tableaux à double entrées : Des notes, des températures, les statistiques d'une saison… En informatique il faut être capable d'enregistrer ces données. Dans cette activité, nous allons voir comment comprendre et organiser des données sous ces formats.

I) DECOUVERTE D'UN FICHIER CSV ET ANALYSE DE SA STRUCTURE :

Connaissez-vous des formats d'enregistrement de données sous forme de table ? ods , .xls, xlsx, .csv etc.
Le format csv est un format simple pour enregistrer des tableaux. Voici la définition de la Wikipédia :

> ***Comma-separated values***, connu sous le sigle **CSV**, est un format texte ouvert représentant des données tabulaires sous forme de valeurs séparées par des virgules.

Un fichier CSV est un fichier texte, par opposition aux formats dits « binaires ». Chaque ligne du texte correspond à une ligne du tableau et les virgules correspondent aux séparations entre les colonnes. Les portions de texte séparées par une virgule correspondent ainsi aux contenus des cellules du tableau. Une ligne est une suite ordonnée de caractères terminée par un caractère de fin de ligne (*line break* – LF ou CRLF), la dernière ligne pouvant en être exemptée.

Voici un exemple de csv et un affichage des mêmes données sous forme de tableau :

```
Sexe,Prenom,Nom,Date_de_naissance
F,Ada,Lovelace,1815
M,Alan,Turing,1912
M,Linus,Torvalds,1969
F,Grace,Hopper,1906
```

Sexe	Prénom	Nom	Date_de_naissance
F	Ada	Lovelace	1815
M	Alan	Turing	1912
M	Linus	Torvalds	1969
F	Grace	Hopper	1906

Exercice 1 :
Vous allez créer un fichier csv « from scratch ».
a) Lancez le logiciel notepad++ qui permet d'écrire un fichier texte.
b) Enregistrez tout de suite le fichier sous le nom best_personnages.csv.
c) Ensuite créez votre csv de façon à écrire la liste de vos personnages (fictifs ou pas) préférés.
Les champs attendus sont (au minimum) Sexe, Nom, Prénom.

Voici un exemple de données :

Sexe	Nom	Prénom
M	Luke	Skywalker
F	Diana	Prince

d) Enregistrer le travail puis ouvrir ce fichier avec LibreOffice **CALC.**
e) Quels réglages d'importation sont proposés ? A quoi correspondent-ils ? Ouvrir le fichier avec une seule option de séparateur cochée.
f) Vérifier que la table obtenue est correcte.
La virgule n'est pas le seul séparateur possible. Il ne serait pas pratique pour un français de stocker des nombres décimaux.

Par exemple le tableau suivant : 1,2,3,4,5 peut être interprété comme | 1 | 2 | 3 | 4 | 5| ou |1,2|3|4,5|

Le séparateur dans le fichier csv peut être :

- la virgule,
- le point virgule ;
- la tabulation
- l'espace

Autre problème, imaginez un fichier csv dont le séparateur est le point-virgule mais que dans les données un texte contienne un point-virgule… Cela pose problème, comment distinguer le séparateur du caractère du texte ?
Il est possible de spécifier (lors de l'ouverture du fichier) que les champs texte sont délimités par des guillemets.

Exercice 2 :
Trouver les séparateurs permettant d'ouvrir les fichiers **Separateur_Inconnu_1.csv** et **Separateur_Inconnu_2.csv**.
Tester alors l'ouverture de ces fichiers dans LibreOffice en choisissant les bons paramètres.

Ouvrir le fichier avec une seule option de séparateur cochée.

Pour le fichier **Separateur_Inconnu_1.csv**, les séparateurs sont des points-virgules.
Pour le fichier **Separateur_Inconnu_2.csv**, les séparateurs sont des tabulations.

On trouve énormément de données sur internet. Une partie de ces données sont publiques, par exemple le site data.gouv.fr recense un grand nombre de données publiques. Ces données sont librement réutilisables.

Exercice 3 :
Afin de découvrir ce qu'est "l'open data", aller sur le site data.gouv.fr. Cliquer sur "documentation". Résumer en quelques lignes ce qui concerne "l'open data".

L'open data, que l'on traduit par « donnée ouverte », est une pratique de publication sous licence ouverte qui garantit un accès libre aux données numériques et autorise leur réutilisation sans conditions techniques, juridiques ou financières. Son intérêt est lié au big data (voir notre dossier Big data, le boom des données numérique).
Ces contenus peuvent être diffusés par des services publics, des associations ou des entreprises privées. Dans un contexte culturel, l'open data répond à une philosophie du partage du savoir avec le plus grand nombre. Dans un contexte politique, l'open data répond à une démarche de transparence et d'implication des citoyens.
Selon la définition établie par la fondation Sunlight, une donnée brute est considérée comme ouverte si elle est : complète, primaire, opportune, accessible, exploitable, non discriminatoire, non propriétaire, libre de droits, permanente, et gratuite.
Publicité
La France, un des pionniers de l'open data
Le droit d'accès aux informations publiques figure dans la Déclaration des droits de l'homme et du citoyen de 1789 à l'article 15 qui stipule que « la Société a le droit de demander compte à tout agent public de son administration ». Si le principe d'un libre accès à l'information publique est inscrit depuis plusieurs décennies dans le droit des grandes nations occidentales (*cf.* loi du 17 juillet 1978 instaurant le droit d'accès à l'information publique en France), c'est bien l'émergence du numérique qui a donné un élan décisif à l'open data. Dans les années 2000, les États-Unis et le Canada ont fait figure de pionniers, et en Europe ce furent le Royaume-Uni et la France.
L'open data et les collectivités territoriales
Selon le classement Open Data Index qui recense l'état d'ouverture des données publiques dans plus de 120 pays, en 2015, l'Hexagone se classait à la dixième position avec un pourcentage d'ouverture de 63 %. En 2011 fut lancé le portail Internet www.data.gouv.fr, qui permet de consulter des données publiques ouvertes issues des services publics, des collectivités territoriales et des entités de droit privé ou public qui assurent une mission de service public. Tout citoyen a également la possibilité de produire, d'enrichir ou de compléter ces données.
Source :
https://www.futura-sciences.com/tech/definitions/internet-open-data-15502/

II) EXPLOITATION DES DONNEES ET RECHERCHE DE DOUBLON :

Nous voulons, de manière simple et rapide, pouvoir répondre à quelques questions. Nous souhaitons aussi vérifier qu'il n'y a pas, dans le fichier proposé, de doublons, c'est-à-dire de ligne en double.

1°) Import du cvs dans python sous forme de liste de listes :

La première étape est de charger notre fichier dans python.
Nous allons utiliser une fonction qui prendra comme paramètre le nom du fichier csv (**situé dans le même répertoire**).
Cette fonction retournera une liste, dans celle-ci, plusieurs listes. Une liste pour chaque ligne du fichier de données.
Nous verrons plus loin une autre fonction qui permettra de retourner une liste de dictionnaires.

```
import csv

def charger_csv(nom_fic):
    """Fonction qui permet d'importer les données du fichier csv sous forme de liste de listes """
    liste = []  # resultat de la fonction
    # ouverture du fichier CSV
    with open(nom_fic,  # nom du fichier
            "r",  # ouverture en lecture
            newline="",  # évite les problèmes de codage du retour à la ligne
            encoding="utf-8-sig"  # permet de forcer la lecture en utf-8 (sig pour "UTF-8 codec with BOM signature")
            ) as csvfile:  # cvsfile est le fichier que l'on vient d'ouvrir
        # création du lecteur csv indiquant le caractère séparateur
        csv_reader = csv.reader(csvfile, delimiter=";")
        for enreg in csv_reader:  # boucle de parcours des enregistrements
            # enreg est une liste de str contenant chaque champ de l ' enregistrement
            # ajout de enreg dans la liste
            liste.append(enreg)
    return liste
```

Tester :

```
liste_medailles = charger_csv("JO_Liste_Medaille_2012-2014.csv")
```

1°) 1 - Manipulation des listes :

Ouvrir le fichier csv dans un éditeur de texte pour vous aider.
Répondre aux questions ci-dessous, en essayant tout d'abord de prévoir la réponse, puis testant la commande.

a) Que doit renvoyer la commande liste_medailles[0] ?

```
['Fédération',
 'Discipline',
 'Epreuve',
 'NOM',
 'Prénom',
 'Sexe',
 'MEDAILLE(S)',
 'Année',
 'Olympique/paralympique']
```

b) Que doit renvoyer la commande liste_medailles[0][3] ?

```
'NOM'
```

c) Quelle commande entrer pour obtenir comme retour : 'LAVILLENIE' ?

```
liste_medailles[2][3]
```

1°) 2 - Les limites de ce type de structures de données :

Pour rechercher la discipline de liste_medailles[1] quelle information supplémentaire doit-on connaître ?
Pour avoir la discipline de liste_medailles[1], il faut connaitre la place (et donc l'index) de la discipline en question.
Cela n'est pas pratique.
Dans le cas présent cet index vaut 1.

```
liste_medailles[1][1]
```

donne :

```
'Athlétisme'
```

2°) Import du cvs dans python sous forme de liste de dictionnaires :

Pour pallier le problème soulevé précédemment, il est possible d'importer les données sous la forme d'une liste de dictionnaires.
Chaque dictionnaire aura une liste de **clés** qui correspondent aux entêtes de notre csv (les données de la première ligne).
Les **valeurs** correspondront aux données obtenues sur chaque ligne.
Exemple : liste[0]['Epreuve'] est égal à '3000m steeple homme'
La première ligne du fichier csv servant à générer nos **clés** est à ignorer lors du parcours des enregistrements.

2°) 1 - Compléter la fonction ci-dessous :

La fonction ci-dessous est incomplète, la compléter.

```
def charger_csv_dict(nom_fic):
    """Fonction qui permet d'importer les données du fichier csv sous forme de liste de dictionnaires """
    liste = []  # resultat de la fonction
    # ouverture du fichier CSV
    with open(nom_fic,  # nom du fichier
            "r",  # ouverture en lecture
            newline="",  # évite les problèmes de codage du retour à la ligne
            encoding="utf-8-sig"  # permet de forcer la lecture en utf-8 (sig pour "UTF-8 codec with BOM signature")
            ) as csvfile:  # cvsfile est le fichier que l'on vient d'ouvrir
        # création du lecteur csv indiquant le caractère séparateur
        csv_reader = csv.reader(csvfile, delimiter=";")
        # permet de sauter la ligne d'entête si pas besoin
        csv_reader.__next__()
        for enreg in csv_reader:  # boucle de parcours des enregistrements
            # enreg est une liste de str contenant chaque champ de l ' enregistrement
            # ajout du score dans la liste
            liste.append(
            {
                'Fédération' : enreg[0],
                'Discipline' : enreg[1],
                'Epreuve' : enreg[2],
                'NOM' : enreg[3],
                'Prénom' : enreg[4],
                # COMPLETER LE CODE A PARTIR DE CETTE LIGNE
                'Sexe' : enreg[5],
                'MEDAILLE(S)' : enreg[6],
                'Année' : enreg[7],
                'Olympique/paralympique' : enreg[8],
            }
            )
    return liste
```

Exécuter ensuite :

```
liste_medailles_dict = charger_csv_dict("JO_Liste_Medaille_2012-2014.csv")
```

2°) 2 - Tester la fonction :

Après avoir compléter la fonction vue en 2°) 1 -, tester la fonction.

a) Que retourne liste_medailles_dict[0]['Epreuve'] ?

'3000m steeple homme'

b) Que doit retourner liste_medailles_dict[7]['MEDAILLE(S)'] ?

'Argent'.

c) Que doit retourner liste_medailles_dict[39]['Olympique/paralympique'] ?

'Paralympique'

3°) Passons aux choses sérieuses :

Faisons le point ! Nous savons importer un fichier csv sous forme :

- de liste de listes,
- de liste de dictionnaires (tableau de p-uplets).

Nous savons aussi afficher une donnée dans ces listes.
Allons plus loin, il est possible d'exploiter ses données via des traitements que nous effectuerons avec python.
Plus tard (l'année prochaine) nous verrons que des langages peuvent servir à exploiter des bases de données relationnelles. (SQL par exemple ...)
Pour les curieux : lien vers l'article wikipedia sur les bases de données relationnelles.

3°) 1 - Recherche de doublons :

Une première tâche est de vérifier que nos données ne contiennent pas de doublons.
Normalement, en utilisant des données depuis data.gouv.fr on peut espérer que nos données soient "propres".
Mais mieux vaut être prudent, on ne sait jamais !
Créer, une fonction qui prend comme entrée une liste et qui renvoie la liste des doublons. Cette liste sera vide s'il n'y a aucun doublon.

```
def liste_doublons(liste):
    """
    fonction qui prend comme entrée une liste
    et qui renvoie la liste des doublons.
    Cette liste sera vide s'il n'y a aucun doublon.
    """
    doublons = []
    for i in range(len(liste)) :
        # Permet d'exclure les doublons déjà trouvés
        if liste[i] not in doublons:
            # On cherche des doublons après l'élément sélectionné
            for j in range(i+1,len(liste)):
                # On ajoute le doublon si celui-ci existe.
                if liste[i] == liste[j]:
                    doublons.append(liste[i])
    return doublons
```

a) L'exemple ci-dessous doit renvoyer ['alice', 'alice', 'david'].

```
liste_doublons(['alice','alice','bob','charles','david','emy','david','fr','alice'])
```

```
On obtient :
['alice', 'alice', 'david']
```

b) Charger le fichier csv qui s'appelle JO_Liste_Medaille_test_doublons.csv. Afficher les éventuels doublons de cette liste.

On ajoute :

```
# Charger le fichier csv qui s'appelle JO_Liste_Medaille_test_doublons.csv
# Affichez les éventuels doublons de cette liste
liste_medailles_doublons = charger_csv_dict("JO_Liste_Medaille_test_doublons.csv")
liste_doublons(liste_medailles_doublons)
```

On obtient :

```
[{'Année': '2012',
  'Discipline': 'Basket-ball',
  'Epreuve': 'Tournoi féminin',
  'Fédération': 'BASKET BALL',
  'MEDAILLE(S)': 'Argent',
  'NOM': 'DUMERC ',
  'Olympique/paralympique': 'Olympique',
  'Prénom': 'Céline ',
  'Sexe': 'F'},
```

```
 {'Année': '2012',
  'Discipline': 'Basket-ball',
  'Epreuve': 'Tournoi féminin',
  'Fédération': 'BASKET BALL',
  'MEDAILLE(S)': 'Argent',
  'NOM': 'DUMERC ',
  'Olympique/paralympique': 'Olympique',
  'Prénom': 'Céline ',
  'Sexe': 'F'},
 {'Année': '2014',
  'Discipline': 'Biathlon',
  'Epreuve': 'Départ en ligne homme',
  'Fédération': 'Ski',
  'MEDAILLE(S)': 'Argent',
  'NOM': 'FOURCADE',
  'Olympique/paralympique': 'Olympique',
  'Prénom': 'Martin',
  'Sexe': 'M'}]
```

c) Vérifier que la liste des médailles ne contient pas de doublons.

On ajoute :

```
# Vérifier que la liste des médailles ne contient pas de doublons.
liste_doublons(liste_medailles_dict)
```

On obtient :

```
[]
```

3°) 2 - Recherches :

Les exercices suivants auront pour but d'effectuer des recherches dans la table liste_medailles_dict.

Compléter la fonction suivante :

```
def nombre_medailles_fede(federation,liste):
    """
    Fonction qui en paramètre prend l'intitulé d'une fédération (voir csv)
    et la liste des médailles sous forme de liste de dictionnaires
    et qui retourne le nombre de médailles pour cette fédération
    """
    compteur = 0
    for enreg in liste :
        if enreg['Fédération'] == federation:
            compteur += 1
    return compteur
```

a) Vérifier que le nombre de médailles en 'Athlétisme' est de 2.

On ajoute :

```
nombre_medailles_fede('Athlétisme',liste_medailles_dict)
```

On obtient :

```
2
```

b) Trouver le nombre de médailles en 'Handisport'.

On ajoute :

```
nombre_medailles_fede('Handisport',liste_medailles_dict)
```

On obtient :

```
83
```

Compléter la fonction suivante :

```
def nombre_medailles(champ,valeur,liste):
    """
```

```
    Fonction qui en paramètre prend l'intitulé le champ cible (par exemple 'NOM'),
    sa valeur cible (par exemple 'FOURCADE')
    et la liste des médailles sous forme de liste de dictionnaires
    et qui retourne le nombre de médailles correspondant
    """
    compteur = 0
    for enreg in liste :
        if enreg[champ] == valeur:
            compteur += 1
    return compteur
```

c) Vérifier que Martin FOURCADE a obtenu 3 médailles au J.O en 2014.

On ajoute :

```
nombre_medailles('NOM','FOURCADE',liste_medailles_dict)
```

On obtient :

```
3
```

d) Déterminer le nombre de médailles d'or de la France lors de ces 2 J.O.

On ajoute :

```
nombre_medailles('MEDAILLE(S)','Or',liste_medailles_dict)
```

On obtient :

```
48
```